

Архитектура наблюдаемости

AI Architect



Проверить, идет ли запись

Меня хорошо видно & слышно?





Фомин Дмитрий

Руководитель отдела архитектуры

- Больше 15 лет опыта работы архитектором
- Участие в разработке архитектуры в очень больших проектах (BSS)

 @advoc_diaboly

 fomin.dmitry@gmail.com

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе #OTUS
AI-Architect-2026-04



Задаем вопрос
в чат или голосом

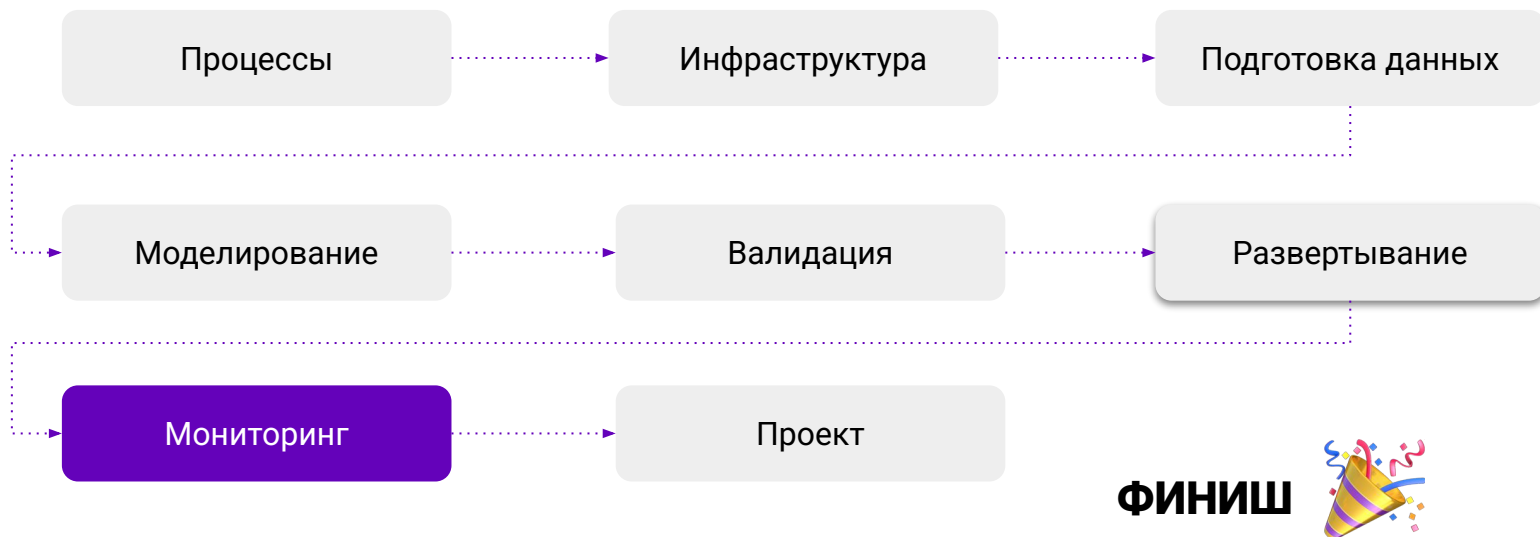


Вопросы вижу в чате,
могу ответить не сразу

Карта курса



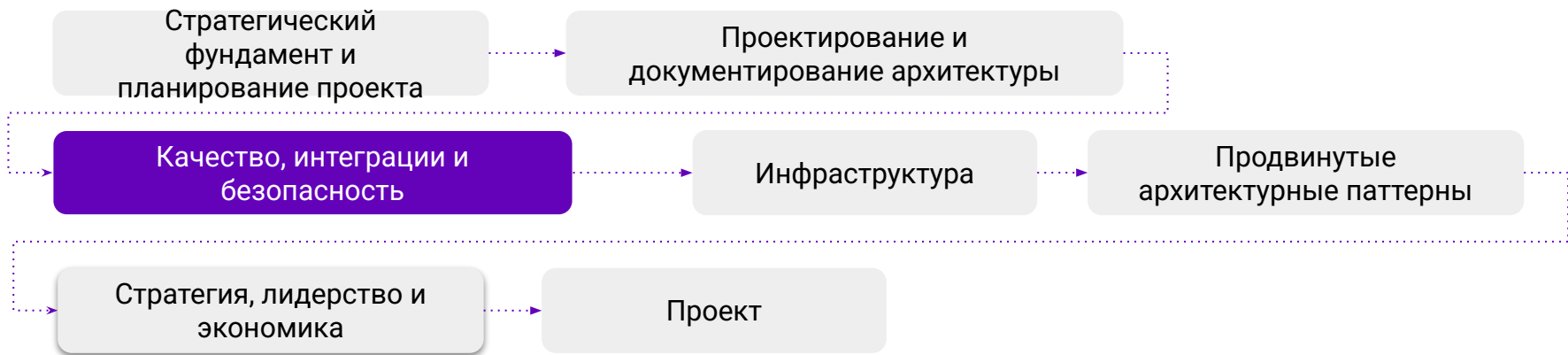
СТАРТ



Карта курса



СТАРТ



ФИНИШ



Маршрут вебинара

Знакомство

Цели и смысл

Наблюдаемость

Алертинг

Практика

Основные тезисы

Цели вебинара

К концу занятия вы сможете

1. Понимать, что включает в себя Observability

2. Понимать, что такое алертинг и правила его настройки

3. Настроить алерт микросервиса с отправкой уведомлений в Telegram



Смысл

Для чего вам это уметь

Чтобы различать метрики, логи, трейсы и знать, как они помогают контролировать ML-системы

Чтобы уметь отличать полезные алерты от шума и выстраивать полезную систему уведомлений

Чтобы на практике уметь контролировать доступность и состояние ML-сервиса и сразу реагировать на сбои

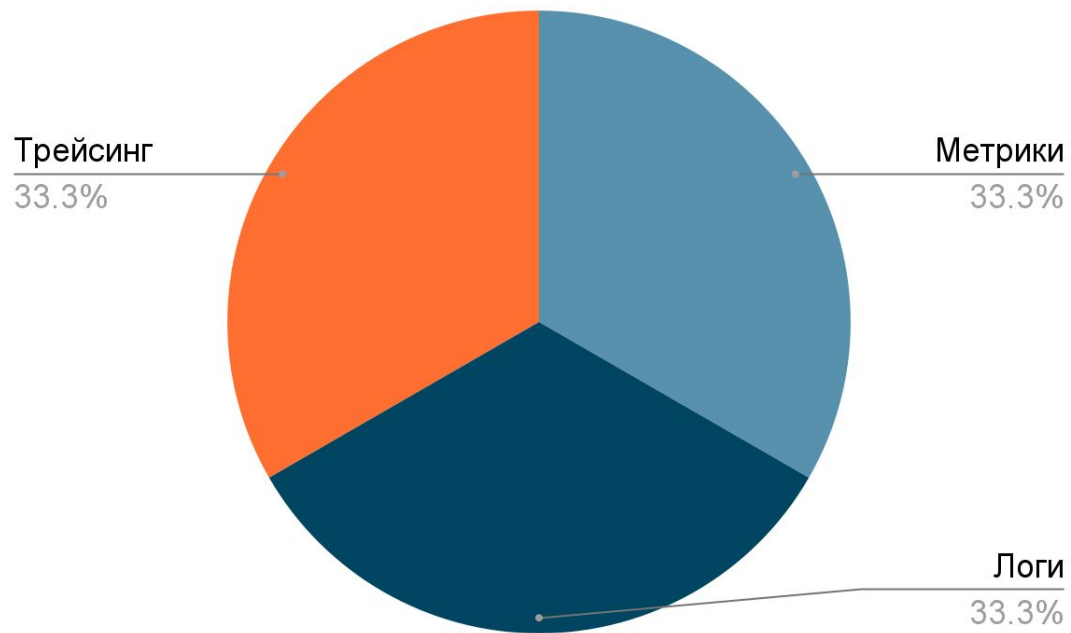


Observability

Observability — это

это способность системы давать достаточно информации о своём внутреннем состоянии, чтобы мы могли понять, что происходит, и быстро находить причины проблем

Компоненты observability



Метрики

Метрики — это числовые показатели, которые описывают состояние системы или ее компонентов

Инструменты для анализа:

- **Prometheus + Grafana**
- **VictoriaMetrics**
- **InfluxDB**



Логи

Логи — это текстовые записи событий, которые происходят в системе (например, ошибки, статусы операций, сообщения от приложений)

Инструменты для анализа:

- **ELK stack (Elasticsearch, Logstash, Kibana)**
- **Loki + Grafana**
- **Syslog**
- **VictoriaLogs**

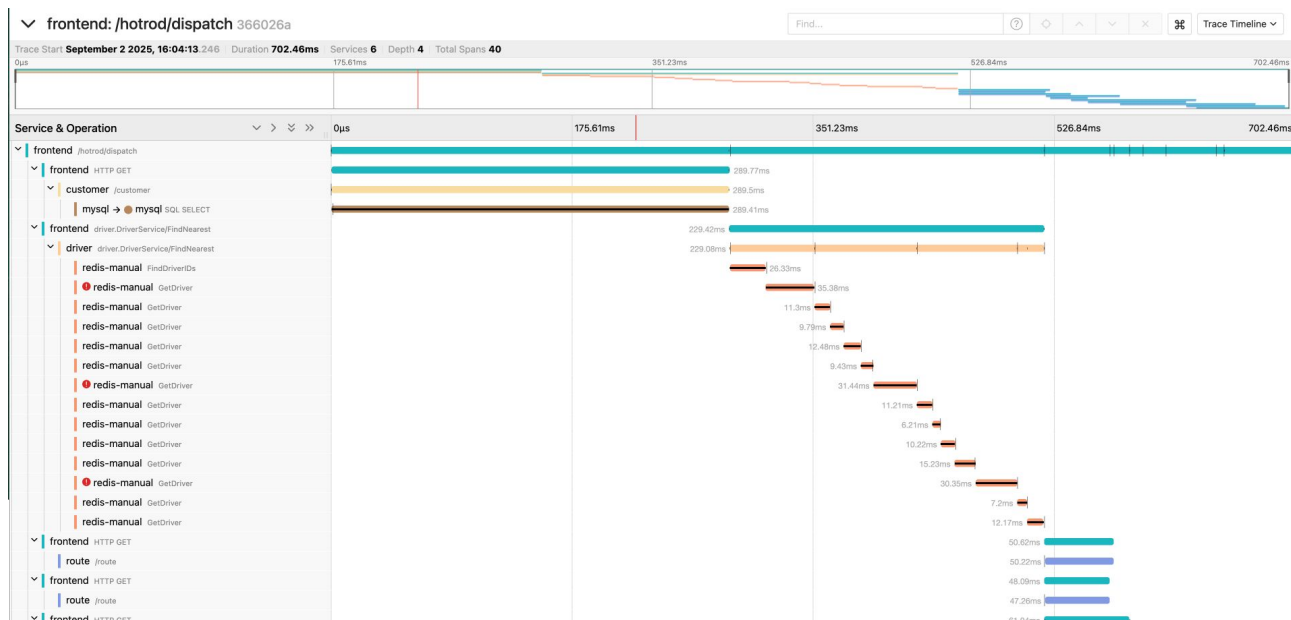
```
Sep 2 11:10:18 10.96.90.114 2025-09-02 11:10:18,961 | INFO | celery.worker.strategy | Task model_inference[163a6ecb-ad64-403d-a1c4-c245c8dce4af] received
Sep 2 11:10:19 10.96.90.114 2025-09-02 11:10:19,076 | INFO | app.services.ml.model_loader | Loading model from MLflow registry: models:/matching/16
Sep 2 11:10:19 10.96.90.114 2025-09-02 11:10:19,098 | INFO | botocore.credentials | Found credentials in environment variables.
Sep 2 11:10:20 10.96.90.114 2025-09-02 11:10:20,182 | INFO | app.services.ml.model_loader | Model version 16 loaded successfully
Sep 2 11:10:20 10.96.90.114 2025-09-02 11:10:20,202 | INFO | app.services.ml.model_loader | Loading preprocessing pipeline from run 01d76c23b8004dea8761943576caa298
Sep 2 11:10:23 10.96.90.114 2025-09-02 11:10:23,184 | INFO | app.services.ml.model_loader | Pipeline feature-pipeline loaded successfully
Sep 2 11:10:23 10.96.90.114 2025-09-02 11:10:23,187 | INFO | app.services.task_runner | Loading data from S3 for file: sessions-data/a176ee539f4a4bbe8f6ca12ca42ec1e5.parquet
Sep 2 11:10:24 10.96.90.114 2025-09-02 11:10:24,530 | INFO | app.services.task_runner | Loading dataframe size: 95928 rows, 24 columns
Sep 2 11:10:24 10.96.90.114 2025-09-02 11:10:24,530 | INFO | app.services.task_runner | Preprocessing data using pipeline
Sep 2 11:10:28 10.96.90.114 2025-09-02 11:10:28,104 | INFO | app.services.task_runner | Filtered dataframe size: 95928 rows, 24 columns
Sep 2 11:10:51 10.96.90.114 2025-09-02 11:10:51,292 | INFO | app.services.task_runner | Preprocessed dataframe size: 95928 rows, 49 columns
Sep 2 11:10:51 10.96.90.114 2025-09-02 11:10:51,292 | INFO | app.services.task_runner | Running model (version: 16) inference
Sep 2 11:10:51 10.96.90.114 2025-09-02 11:10:51,497 | INFO | app.services.task_runner | Result dataframe size: 95928 rows, 3 columns
Sep 2 11:10:51 10.96.90.114 2025-09-02 11:10:51,497 | INFO | app.services.task_runner | Saving results to S3: sessions-data/a176ee539f4a4bbe8f6ca12ca42ec1e5_results.parquet
Sep 2 11:10:51 10.96.90.114 2025-09-02 11:10:51,721 | INFO | celery.app.trace | Task model_inference[163a6ecb-ad64-403d-a1c4-c245c8dce4af] succeeded in 32.75665568560362s: 'sessions-data/a176ee539f4a4bbe8f6ca12ca42ec1e5_results.parquet'
```

Трейсинг

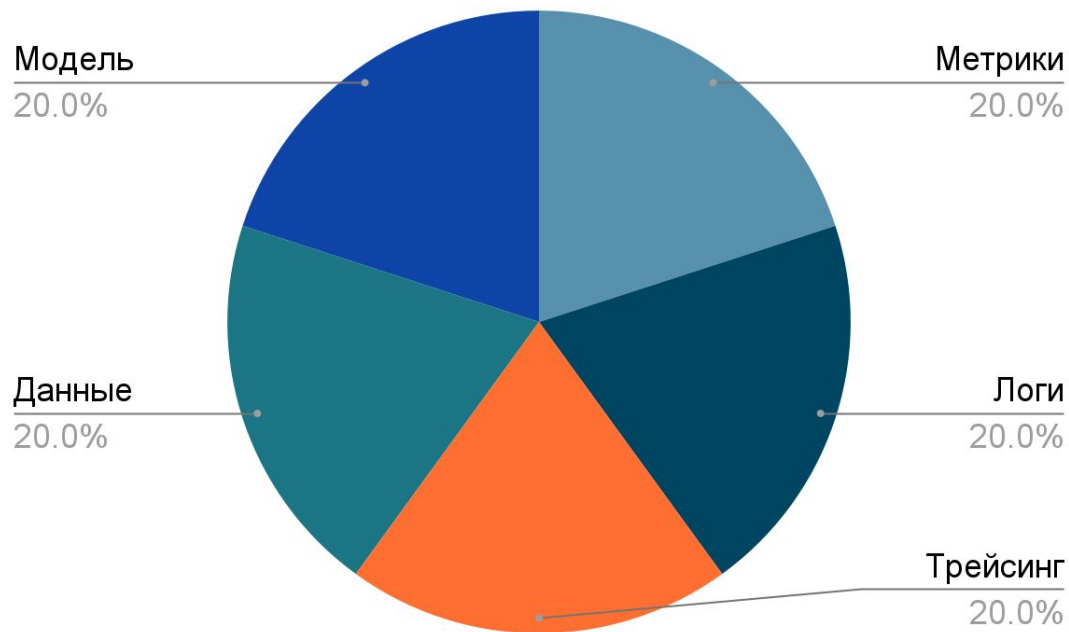
Трейсинг — это сбор информации о том, как запрос проходит через разные сервисы, включая время каждого шага

Инструменты для анализа:

- Jaeger
- Zipkin
- OpenTelemetry



Компоненты observability в ML



Компоненты observability в ML

В ML системах observability включает контроль качества данных и моделей, а не только инфраструктуры

- **Evidently AI, WhyLabs, Arize** — мониторинг качества данных и моделей
- **MLflow, ClearML, Weights & Biases** — отслеживание экспериментов и метрик моделей
- **Langfuse** — инструмент для наблюдаемости LLM-приложений, включающий трассировку, метрики качества и аналитику промптов



Вопросы



если есть вопросы



если вопросов нет

Ключевые тезисы блока

1. Observability = метрики, логи, трейсы
2. В ML критично наблюдать и за инфраструктурой, и за приложением, и за данными, и за моделью



Алертинг

Алертинг — это

сигнал для действия, автоматически формируемый
при выполнении условия над телеметрией
(метрики, логи, трейсы, проверки данных)



Метрика ≠ Алерт: метрика — сырые измерения

Алерт = метрика(и) + условие + приоритет + доставка

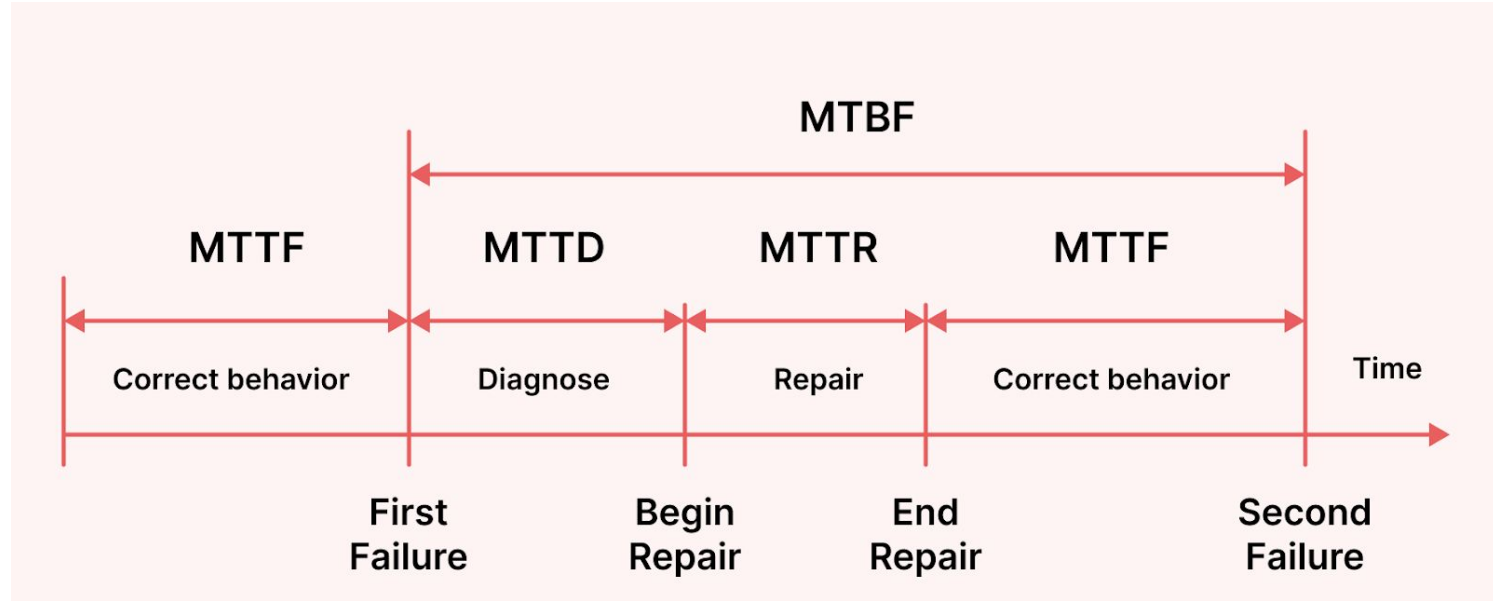


Для чего нужны алерты

1. Ранняя реакция на сбои
2. Поддержка SLA/SLO
3. Защита бизнес-метрик
4. Сокращение времени обслуживания системы
5. Автоматизация и self-healing
6. Предотвращение проблем заранее
7. Поддержка командной работы и эскалации



Время обслуживания системы



Доступность сервиса

Доступность — это характеристика, показывающая, какую часть времени сервис был работоспособен

Доступность = (Согласованное время обслуживания – Время простоя) / Согласованное время обслуживания * 100

Availability %	Downtime hours per year
99.8	17.52
99.9	8.76
99.95	4.38
99.99	0.876
99.999	0.087

Состав алерта

1. Сигнал(-ы)
2. Правило/условие
3. Контекст и метаданные
4. Маршрутизация
5. Runbook



Типы правил для алертов

1. Threshold
2. Rate of change
3. Absence/Heartbeat
4. Anomaly
5. Composite



Классификация алертов

По severity сервиса

- Critical
- Error
- Warning
- Info

По источнику алерта

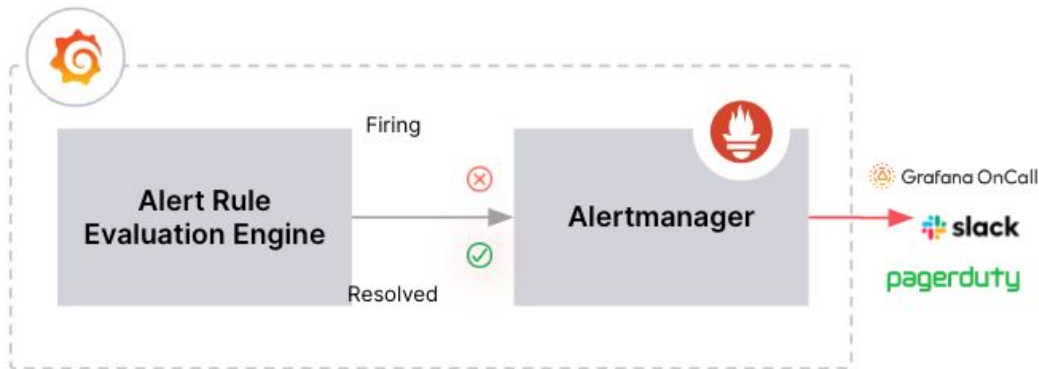
- Микросервисы
- Базы данных
- Очереди/топики
- ML данные

По severity системы

- Mission critical
- Business critical
- Business operational
- Office Productivity

Системы управления алертами

- Prometheus
- Grafana
- Sentry
- Prometheus Alertmanager
- Grafana Alerting
- Opsgenie / PagerDuty / VictorOps



Примеры метрик для алертов



Инференс/онлайн-сервисы

- Latency
- CPU/GPU/RAM/Disk Usage
- Error Rate
- Request Rate



Пайплайны и очереди

- Orchestration SLA
- Kafka/Queues: consumer lag
- Batch freshness



Данные и модель

- Качество модели
- Дрейф данных/признаков
- Стабильность предсказаний
- Валидация данных



Трейнсинг для LLM

- Рост доли неуспешных tool/function-calls
- Таймауты шагов в цепочке
- Рост стоимости/токенов на запрос

LLM метрики

- **Метрики качества**

- a. Relevance
- b. Correctness
- c. Completeness
- d. Consistency
- e. Style

- **Метрики RAG**

- a. Context Relevance
- b. Context Coverage
- c. Faithfulness
- d. Retrieval метрики

- **Метрики для чат-ботов**

- a. Task Success
- b. Диалоговые метрики

- **Метрики безопасности и соответствия политикам**

- a. Токсичность
- b. PII
- c. Policy violations

- **Мониторинг деградаций**

- a. Query drift
- b. Response drift
- c. Retrieval drift

LLM метрики

Инструменты



Microsoft Presidio

Правила хорошего алерта



1. Actionable
2. Sensible severity
3. Alert fatigue
4. Ясность и контекст
5. Привязка к SLO/бизнес-метрикам
6. Testable

Что делать если пришел алерт



Что делать

1. Уведомить заказчика/бизнес о проблеме
2. Проверить логи, метрики, трейсы, дашборды
3. Проверить чаты/алерты смежных систем
4. Начать устранение проблемы
5. Написать постмортем и закрыть его (с исправлениями)
6. Если алерт оказался бесполезным, то удалить его
7. Если алерт полезный, то похвалить себя

Что не делать

- Искать кто починит вместо действий
- Игнорировать или откладывать алерт
- Решать ложный/неложный без проверки
- Сообщать, что починили, если нет 100% уверенности

Вопросы



если есть вопросы



если вопросов нет

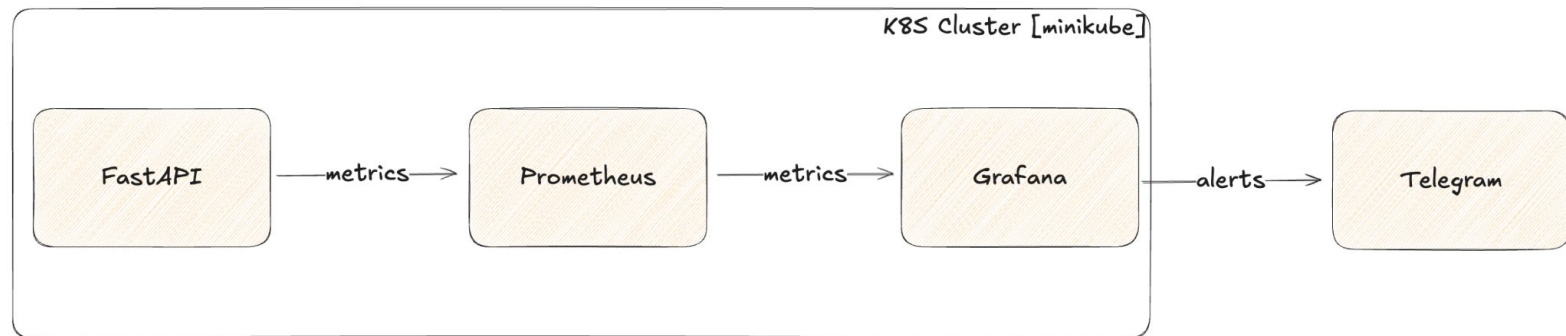
Ключевые тезисы блока

1. Алерты = метрики + правила + уведомления → actionable сигналы
2. Классификация: по severity, источнику, критичности системы
3. Алерты помогают вовремя реагировать и защищать SLA и бизнес-метрики
4. Сначала починить, потом обсуждать
5. Регулярно проводите ревью ваших алертов



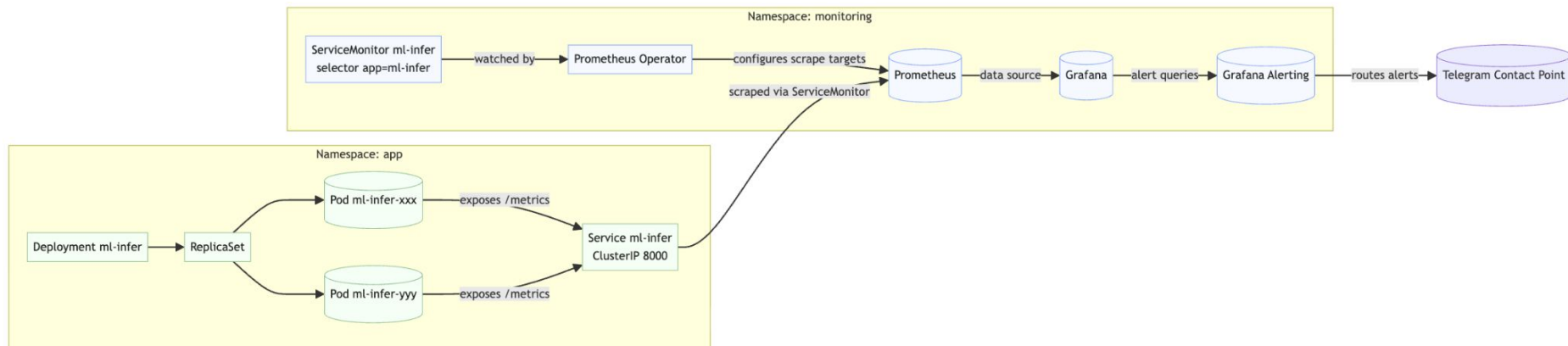
Пример

Что сделаем



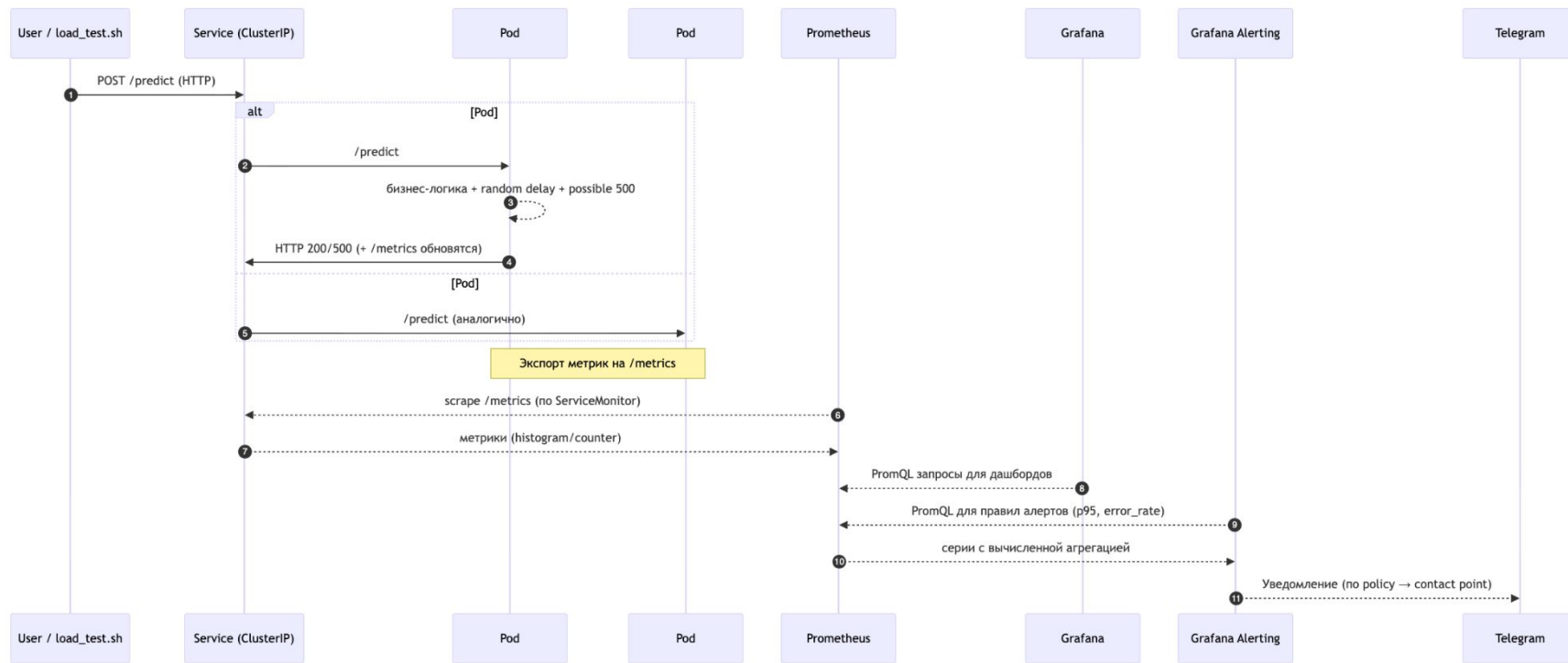
Что сделали

Топология в Kubernetes



Что сделали

Путь данных от запроса до алерта



Вопросы



если есть вопросы



если вопросов нет

**Подведём
итоги занятия**

Ключевые тезисы занятия

1. Алертинг строится на метриках, но превращает их в actionable сигналы
2. Хорошие алерты снижают шум и помогают фокусироваться на реально важных проблемах
3. Алерты и наблюдаемость связывают технические события с бизнес-метриками
4. Культура работы с инцидентами (postmortem, пересмотр алертов) делает систему надёжнее со временем



Теперь вы сможете

1. Понимать, что включает в себя Observability

Чтобы различать метрики, логи, трейсы и знать, как они помогают контролировать ML-системы

2. Понимать, что такое алертинг и правила его настройки

Чтобы уметь отличать полезные алерты от шума и выстраивать полезную систему уведомлений

3. Настроить алерт микросервиса с отправкой уведомлений в Telegram

Чтобы на практике уметь контролировать доступность и состояние ML-сервиса и сразу реагировать на сбои



Список материалов для изучения

1. Статья: [Классификация критичности информационных систем](#)
2. Статья: [Человеческим языком про метрики 2: Prometheus](#)
3. Статья: [Что такое цель по уровню обслуживания \(SLO\)?](#)
4. Документация: [Configure Alertmanager](#)
5. Книги: [Google SRE books](#)
6. Практика: [Jaeger live demo](#)



Домашнее задание

ДЗ: Комплексное обеспечение качества

- Шаги выполнения:
 - Security Layer: Добавьте на архитектурную схему компоненты защиты (PII Sanitizer - скрывание персональных данных перед отправкой в LLM, Guardrails - валидация ответов модели).
 - Testing Strategy: Опишите, как будете оценивать качество RAG (метрики Faithfulness, Answer Relevancy). Каким инструментом (Ragas, DeepEval)?
 - Observability: Спроектируйте дашборд для Grafana. Перечислите 4-5 ключевых графика (Golden Signals: Latency, Traffic, Errors, Saturation) + специфичные AI-метрики (Token usage, Average Cost per Request).
- Формат сдачи:
 - Документ с описанием:
 - Схемы Security;
 - Плана тестирования;
 - Мокапа/Списка виджетов дашборда.



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

Приглашаем на следующие вебинары по проектной работе



Ссылка на вебинар будет
в ЛК за 15 минут



Материалы
к занятию в ЛК —
можно изучать



Обязательный материал
обозначен красной лентой

**You are our
OTUS heroes**

